

CPMS

One

Britt Anderson

Winter 2025

Contents

1	Reviewing Last Week's Exercises and Assignments	1
2	Some Additional Questions For Computational Cognition:	1
3	What Is Cognition?	2
4	Cognition is ... ?	2
5	Is Cognition Computational?	4
5.1	Introduction	4
5.1.1	Computing in Minds and Computers	4
5.1.2	Classical Computational Theory of Mind	6
5.1.3	Programming a Turing Machine: the Busy Beaver	7

1 Reviewing Last Week's Exercises and Assignments

1. Reading Discussions
2. Sharing Posters
3. Revisiting some of our key questions from last week.

2 Some Additional Questions For Computational Cognition:

1. Is the brain a computer?
2. What does it mean for something to be computable?
3. What is the computational theory of mind, and what does it mean for us practically?

3 What Is Cognition?

The technical challenges for modeling in psychology are mastering the mathematics and programming necessary to formally express an idea and to implement a simulation to examine the consequences. The conceptual challenge is to decide what it is you are modeling. And it appears that there is no consensus on what is cognition.¹

In the late 1800s Ebbinghaus, using creative, and for their time, innovative, Herculean methodologies, demonstrated that the proportion of items retained in memory declines predictably over time. The form of the forgetting curve is exponential.

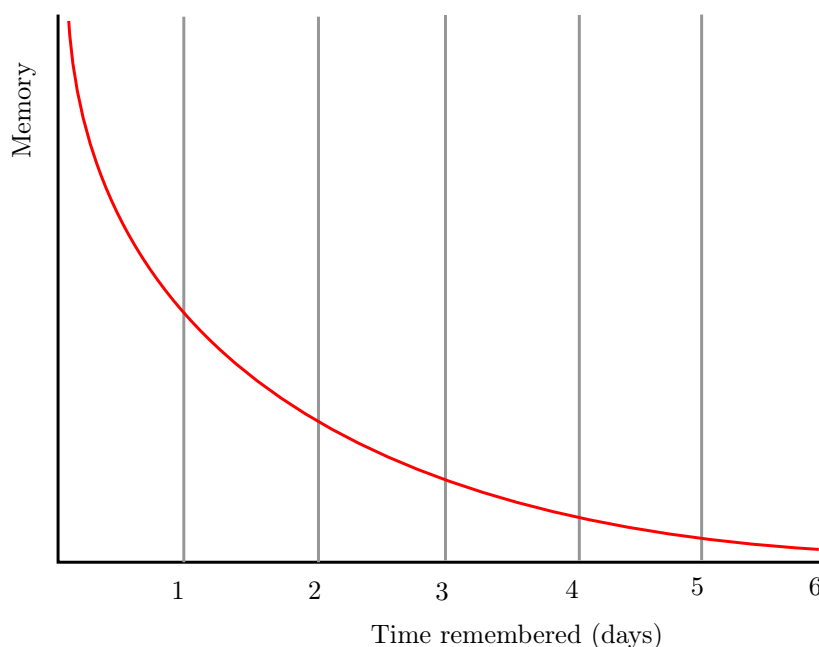


Figure 1: By The original uploader was Icez at English Wikipedia.

What can we infer from the similarity between Figure 2 and Figure 1?

4 Cognition is ... ?

As revealed in² there are a wide array of views on what is cognition.

¹Tim Bayne et al., "What Is Cognition?," *Current Biology* 29, no. 13 (2019): R608–15, <https://doi.org/10.1016/j.cub.2019.05.044>.

²Bayne et al.

Which of these opinions, if any, do you agree with and why?

Is a mathematical formula that reproduces an observed behavioral pattern a model?

Which of these opinions, if any, do you agree with and why?

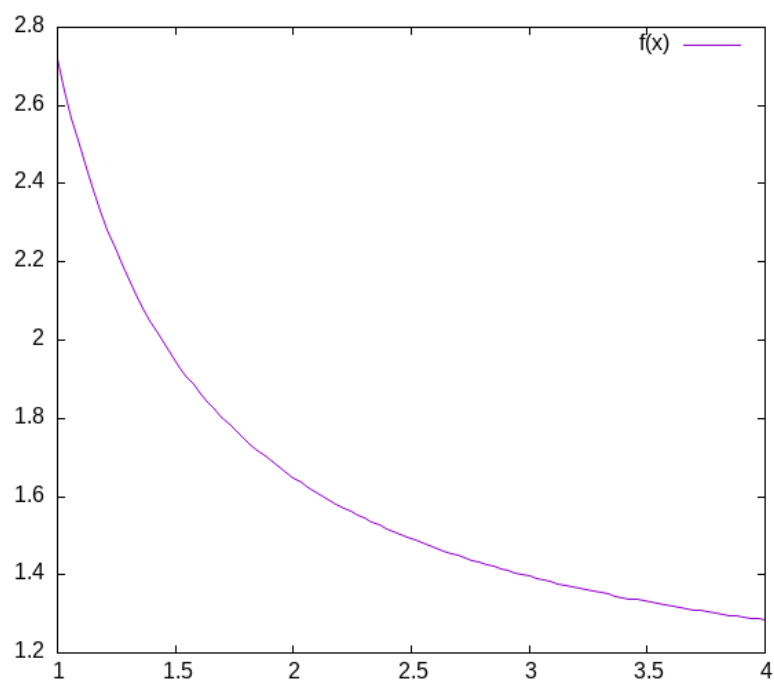


Figure 2: Plot of $e^{\frac{1}{x}}$.

5 Is Cognition Computational?

1. Is there anything a human can think that a computer cannot compute?
2. What does it mean for a function to be "computable"?
3. What does it mean to say that cognition is computational?
4. What implications does the answer have for modeling cognition?

5.1 Introduction

Is there a difference between computational cognitive neuroscience and cognitive computational neuroscience? The former seems to suggest that we are interested in explaining cognition directly from the actions of neurons and then using computational tools as adjuncts to aid this attempt. On the other hand, when you reverse the order it seems as if you are trying to explain thinking via the computational accounts of neuronal function. The latter seems to require a clear link between notions of computation and models of thinking. To use Marr's three levels as a scaffold³; we are taking neurons as our implementation, positing algorithms for them, and assuming that computational principles populate the level of our cognitive abstractions. Measures of success depend on the meanings of terms.

Behind all of this is the idea that if you want to understand *mind* you need to first develop a *theory*. Explanations offered in words only are really more proto-theories than theories because of their imprecision. To express a theory clearly and unambiguously you need to *formalize* it. Either you express it as *math* or you can *code* it. Let's explore the distinction between computation as implementation and computation as the basis for mind.

5.1.1 Computing in Minds and Computers

One definition of whether something is computable is whether there exists a Turing Machine that can compute it. Although most of us learned the name "Turing" in the context of whether a computer has human intelligence, the so-called *Turing test*, the model of Turing computability emerged from thinking about humans computing⁴

1. What is a Turing machine? Some Background and Details Turing machines are quite simple implements. While one can build a physical Turing machine, the more usual sense of the term is for a hypothetical computer that is comprised of:
 - (a) a finite alphabet,

³Ron McClamrock, "Marr's Three Levels: A Re-Evaluation," *Minds and Machines* 1, no. 2 (May 1991): 185–96, <https://doi.org/10.1007/bf00361036>.

⁴Alan Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem (1936)," in *The Essential Turing* (Oxford University Press/Oxford, 2004), 58–90, <https://doi.org/10.1093/oso/9780198250791.003.0005>.

This reference is an older one, and not the original, but it does use LISP as its example, which is the classic example of good old fashioned AI. What are Marr's three levels?

A nice short version of this history can be found in [\[\[https://link.springer.com/chap/10.1007/978-1-4419-3642-3_642-31933-4.pdf\]\]](https://link.springer.com/chap/10.1007/978-1-4419-3642-3_642-31933-4.pdf)[pdf]

- (b) a finite set of states,
 - (c) the capacity to read and write to a single location in memory, and the ability to adjust the memory location immediately left or right or to make no move at all, and
 - (d) a set of instructions (or "machine table") that translates the combination of the current state and the current symbol to a new state and one of the acceptable actions.
- (a) Classroom Activity Let's develop pseudo-code for the implementing a Turing Machine

2. The Lambda Calculus

Another model of computation is the *lambda calculus*⁵.

The lambda calculus was developed as a theory of functions. John McCarthy invented the computer programming language Lisp as a theoretical exercise for working on the theory of computable functions. He felt the Turing machine was too mechanical and too awkward for this sort of theoretical work. He wanted a better tool; a better metaphor. He adopted the lambda of the lambda calculus and the `eval` function to take in lisp programs and execute them. Later one of his collaborators observed that it was relatively straightforward to implement this theoretical language as a real programming language. A bit more of the history this development can be found here. To get more details see.⁶

- (a) Some Details and Interesting Facts about the Lambda Calculus
 - i. There is not just one lambda calculus.
 - ii. To have the lambda calculus you need to specify your algebra. What are the rules for reducing things (i.e. your computations), what are the allowed symbols, and what do things mean? A "lambda calculus" is a way of handling "lambda expressions."⁷

(b) Into the Weeds

Terms are either *simple variables* e.g. x or y or *composite terms* like $\lambda v . t_1$. Having two terms next to each other $t_1 t_2$ means "apply" t_1 to t_2 . The meaning of a term like $\lambda v . t_1$ is the value returned by the lambda abstraction. The meaning part is sometimes designated in writing by formulas using arrows, such as $t_1 \rightarrow t_2$.⁸

Some "axioms" you supply, or that Alonzo Church did are:

⁵And yet another is the *theory of recursive functions*

⁶Greg Michaelson see *An Introduction to Functional Programming through Lambda Calculus* (Mineola, N.Y: Dover Publications, 2011), <https://www.cs.rochester.edu/~brown/173/readings/LCBook.pdf>.

⁷The λ of lambda calculus doesn't really mean anything. It just signals that you have a lambda expression. Its creation as the symbol for the function calculus was an accident of notation and the limits of older typewriters.

⁸Want to try implementing the lambda calculus yourself? Here is a blog post on doing this in the computer language Common Lisp.

- Beta reduction
- Beta abstraction
- Alpha conversion
- Eta reduction
- Normal Form

The equivalent of the halting problem for the Turing machine is the reaching of a *normal form* in the lambda calculus.

Functions have the form $\lambda \langle \text{name} \rangle . \langle \text{body} \rangle$. Note the "dot". This separates the name from the body of expressions that it names. $\langle \text{body} \rangle$ is also an expression (note the recursion that is built in).

What are recursive functions?

(c) Some Opportunities for Practice

- Write the lambda expression for the identity function?
 - But first we better decide what the identity function is?
- Apply the identity function to itself.
- What is the identity function in whatever language you are hoping to use?
- An interesting lambda expression is the "self-application" expression: $\lambda s.(s\ s)$.
 - With pencil and paper try to:
 - * apply this to the identity,
 - * apply the identity to the self-application,
 - * apply the self-application to itself. What is its termination status?

IAMHERE

but it appears to be the case that they are equivalent. Anything designated "computable" by one of these formal accounts is computable by the others as well. Does this mean that if you accept the computational mind hypothesis you must accept that the mind is able to be simulated by a Turing Machine? As a consequence does this mean that minds are multiply realizable?

- Are Turing machines /digital?
- Is this an important distinction?
- Does this mean that analog computations are omitted?
- Would an analog paradigm be a better match to modeling mental activity?

5.1.2 Classical Computational Theory of Mind

The classical computational theory of mind says that in all important ways the mind is like a Turing machine. If we want to model (or emulate) functions of mind one way would be to build a Turing machine. However this can be a practical challenge, and one can question the insight gained from this approach. Still

given the theoretical and practical prominence given to Turing computability it behooves us to know what a Turing machine truly is.

That Turing machine must halt. Since one cannot decide in advance for all machines whether they will halt or not it turns out that whether or not something is computable is, in general, undecidable.

5.1.3 Programming a Turing Machine: the Busy Beaver

The following is a slightly formatted version of the Wikipedia description of a Turing machine.

1. A Turing machine has n "operational" states plus a Halt state, where n is a positive integer, and one of the n states is distinguished as the starting state.}
2. The machine uses a single two-way infinite (or unbounded) tape.}
3. The tape alphabet is $\{0, 1\}$, with 0 serving as the blank symbol.}
4. The machine's transition function takes two inputs:
 - (a) the current non-Halt state,}
 - (b) the symbol in the current tape cell,}}
 - (c) and produces three outputs:
 - i. a symbol to write over the symbol in the current tape cell (it may be the same symbol as the symbol overwritten),}
 - ii. a direction to move (left or right;" that is, shift to the tape cell one place to the left or right of the current cell), and}
 - iii. a state to transition into (which may be the Halt state).